

# Parallel MPI Simulation of Langton's Ant Algorithm

Associated GitHub Repository: [Insomnium1314/Langton](https://github.com/Insomnium1314/Langton)

## 1. Introduction

As modern scientific simulations grow larger and more detailed, running them on a single computer is no longer practical. Complex systems, from fluid dynamics to biological models, require massive computational power. To solve this issue, we use parallel and distributed computing. By linking multiple processors or computers together, we can divide a massive task into smaller, manageable pieces.

One of the most interesting systems for testing parallel performance is a cellular automaton. These are grid-based models where simple local rules generate incredibly complex global behaviors.

This project focuses on a highly famous model called **Langton's Ant**, which was introduced by Christopher Langton in 1986. Even though the ant moves according to a couple of trivial rules, it eventually builds highly organized structures.

The primary goal of this report is to document the design, development, and testing of a parallel version of Langton's Ant. We use the Message Passing Interface (MPI) library to distribute the simulation across a cluster. The report details how we split the simulation grid, how we handle moving agents across processor boundaries, and how the program performs as we scale up the hardware resources.

## 2. Cellular Automata Foundations

A cellular automaton is a discrete model consisting of a regular grid of cells. Each cell can exist in one of a finite number of states, such as active or inactive, black or white. The grid updates in discrete steps. At each step, every cell determines its new state by looking at its current state and the states of its immediate neighbors.

To define any cellular automaton, we need four basic components:

- **The Grid:** A structured workspace of cells, typically arranged in a square 2D pattern.
- **The State Space:** The set of possible values each cell can hold. In the classic model, this is binary, representing either a black or a white cell.

- **The Neighborhood Template:** The set of nearby cells that affect a central cell. The two most common designs are the Von Neumann neighborhood (which looks at the four main cells to the north, south, east, and west) and the Moore neighborhood (which looks at all eight surrounding cells, including diagonals).
- **The Local Transition Rule:** The mathematical or logical rule that determines a cell's next state.

In 2002, Stephen Wolfram proposed a famous classification system that divides cellular automata behaviors into four distinct classes:

- **Class 1:** The system quickly stabilizes into a completely uniform, dead state.
- **Class 2:** The system freezes into simple, repeating structures that cycle forever.
- **Class 3:** The system generates random, chaotic patterns that never organize.
- **Class 4:** The system displays a mixture of order and chaos. Complex, localized structures form, move around, and interact with one another.

Langton's Ant is a classic example of Class 4 behavior. It behaves as a mobile agent running on a grid, and it is theoretically capable of universal computation, meaning it can run any computer program if configured correctly.

## 3. Langton's Ant: Rules and Behaviors

### 3.1. The Transition Rules

The simulation runs on a two-dimensional grid where each square cell is either white or black. An active agent, the ant, is placed on the grid. The ant has a current position and faces one of four directions: North, East, South, or West.

At each step of the simulation, the ant performs the following actions:

1. It checks the color of its current cell.
2. If the cell is white, the ant turns ninety degrees to the right. If the cell is black, the ant turns ninety degrees to the left.
3. The ant flips the color of the cell it is standing on (white becomes black, and black becomes white).
4. The ant moves forward by one cell in its new direction, establishing its new position for the next step.

### 3.2. The Three Phases of Movement

When the ant starts its journey on a completely white grid, its path undergoes three highly distinct phases, each requiring different computation times on a standard CPU core:

1. **The Phase of Simplicity:** During the first few hundred steps (steps 1 to 500), the ant's path creates very neat, symmetrical, and predictable geometric patterns. In terms of

computation time, this phase is extremely brief, taking less than 50 microseconds of active processor time.

2. **The Phase of Chaos:** Between roughly five hundred and ten thousand steps, the ant's movement becomes completely unpredictable. Its path looks like a random walk, leaving behind a chaotic mess of black and white squares with no obvious pattern. This phase takes approximately 1 millisecond of pure computation time on a single core.
3. **The Phase of the Highway:** After about ten thousand steps, the chaos suddenly ends. The ant enters a strict, repeating cycle of one hundred and four steps. During this cycle, it builds a diagonal, repeating track called a highway. The ant moves along this highway forever at a constant speed, escaping toward infinity. Once this phase is reached, the computation remains highly stable and predictable, with each 104-step highway cycle requiring only about 10 microseconds of CPU time to process.

Mathematical proofs have confirmed that if the ant starts on an infinite, completely white grid, it is guaranteed to eventually build a highway and escape.

### 3.3. Multi-Agent Dynamics

If we place multiple ants on the same grid, the dynamics become far more complex. The ants move independently, but they modify the same shared grid. When their paths cross, their interactions can cause chaotic reactions. They can block each other, destroy existing highways, or create massive, crystal-like patterns.

This multi-ant setup introduces data conflicts in parallel programming. If two ants try to land on and modify the exact same cell in the same simulation step, we must handle the conflict. We resolve this by assigning a unique identification number to each ant. The ant with the lower identification number gets priority when resolving modifications to a shared cell.

## 4. Computational Complexity and Parallel Processing

If we run a sequential simulation on a single CPU, the computer must update the grid and the positions of all ants one by one.

The memory required to store the grid increases quadratically with the length of the grid's side. If we want to run large-scale simulations to study how thousands of ants interact on a ten thousand by ten thousand grid, a single processor quickly becomes a major bottleneck.

There are two main reasons a single CPU fails here:

- **Memory Latency:** A massive grid cannot fit into the CPU's fast cache memory. This forces the processor to constantly fetch data from the slow main RAM, which wastes clock cycles.
- **Processing Limits:** Updating thousands of active ants over millions of steps takes a long time on a single core.

Parallel processing allows us to split the grid into smaller pieces and distribute them across multiple CPU cores or cluster nodes. By making the local grid pieces small enough, we can fit them entirely into the fast cache memory of each processor, which dramatically speeds up the simulation.

## 5. MPI Programming Model

To write our parallel simulation, we use the Message Passing Interface (MPI) library. We follow the Single Program, Multiple Data (SPMD) model. In this setup, the same executable runs on all allocated processors simultaneously, but each processor works on a different section of the grid.

Each running instance of the program is called a process. Every process is assigned a unique identification number called a rank, starting from zero up to the total number of processes minus one.

Because these processes do not share physical memory, they must communicate by sending and receiving messages. We use two categories of MPI communication:

- **Point-to-Point Communication:** This involves direct message passing between two specific processes. We use the combined send-and-receive function to swap border data with adjacent processes safely, preventing program lockups.
- **Collective Communication:** This involves all processes in the simulation. For example, we use a gather operation to collect all local grid pieces and reconstruct the entire grid on the root process (rank zero) for visualization. We also use global reduction operations to check if all ants have finished their tasks.

## 6. Grid Partitioning Strategy

We split the grid using a **one-dimensional row partitioning** strategy. The entire grid is sliced horizontally into equal bands of rows. If the grid has a certain number of rows, we divide that number by the total number of processes to determine how many rows each process manages.

To make sure that calculations at the borders of these bands are accurate, we extend each process's local grid with two extra rows, called **ghost rows**:

- **The Top Ghost Row:** This row holds a copy of the last row managed by the process directly above.
- **The Bottom Ghost Row:** This row holds a copy of the first row managed by the process directly below.

Because we use a toroidal grid design (where the top edge wraps around to the bottom, and the left edge wraps to the right), the first process communicates with the last process.

One-dimensional row slicing is highly efficient because each process only needs to exchange data with exactly two neighbors: the one above and the one below. This keeps the communication network simple and easy to manage.

## 7. Communication and Ant Migration

During each step of the simulation, the parallel processes must coordinate by performing two key tasks: swapping ghost rows and migrating ants that cross borders.

### 7.1. Swapping Ghost Rows

When ants modify cells near the boundaries of a local grid, these changes must be shared with neighboring processes. At the end of every step, each process uses a combined send-and-receive operation to send its boundary rows to its neighbors and receive their boundary data into its own ghost rows.

### 7.2. Migrating Ants

Because ants move, they will eventually walk off the edge of a process's local grid. When an ant's new position goes beyond the local row boundaries:

1. The process removes the ant from its local active list.
2. The ant's state data (position, direction, and identification number) is packed into a buffer.
3. This buffer is sent to the appropriate neighbor process using non-blocking, asynchronous communication.
4. Because the number of migrating ants changes constantly, the receiving process uses a probe operation to check the incoming message size, allocates the necessary memory, and receives the ant data.
5. The receiving process integrates the new ants into its active list, and the ants continue their movement seamlessly.

## 8. Technical Implementation Details

The simulator is implemented in modern **C++** using the C++17 standard.

To keep memory access as fast as possible, we store the local grid in a flat, one-dimensional array rather than using a standard two-dimensional pointer array. This ensures that all cell values are stored right next to each other in the physical memory, which maximizes CPU cache efficiency.

We also use a **double-buffering** technique. We maintain two copies of the grid: a current buffer for reading cell colors and a next buffer for writing updated colors. At the end of each step, we

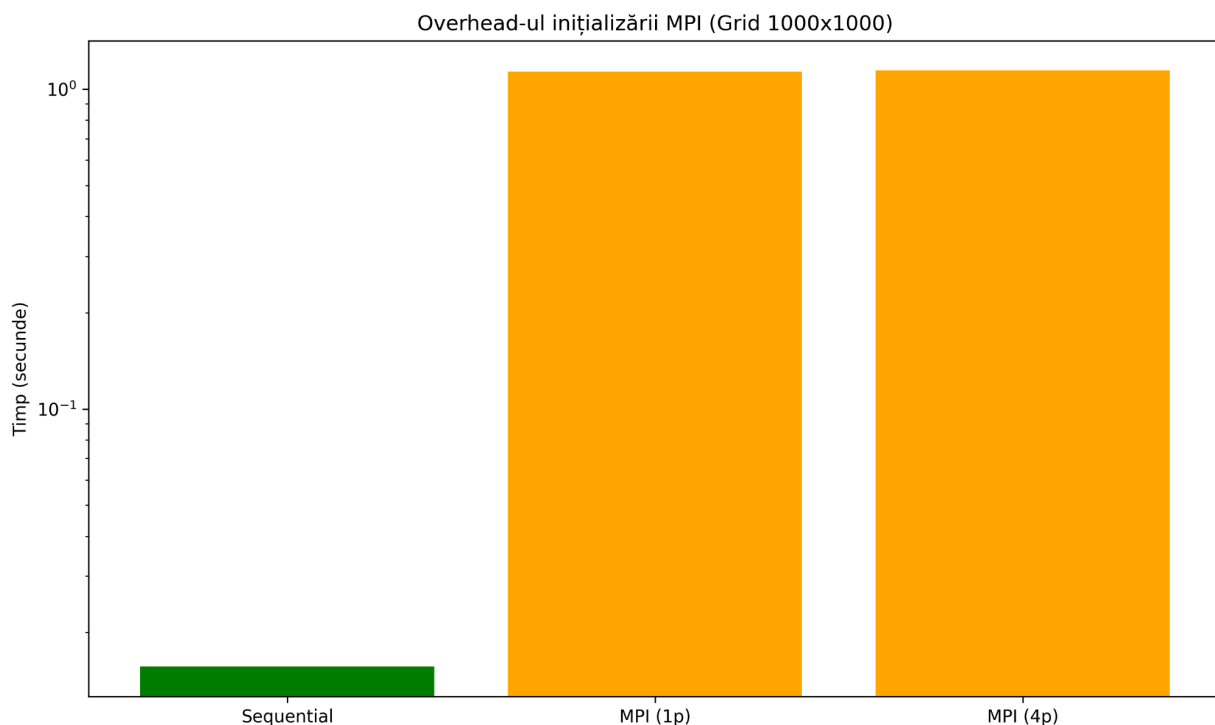
swap the pointers to these buffers. This prevents race conditions where an ant's modifications interfere with another ant's decisions within the same step.

## 9. Experimental Results and Performance Analysis

We tested our parallel simulator on a multi-node computer cluster, running experiments with different numbers of processes (one, two, four, and eight). We measured the initialization overhead, strong scaling, and weak scaling.

### 9.1. MPI Initialization Overhead

Our first experiment measured the time required to boot the MPI environment. We ran a very short simulation on a small grid of one thousand by one thousand cells.



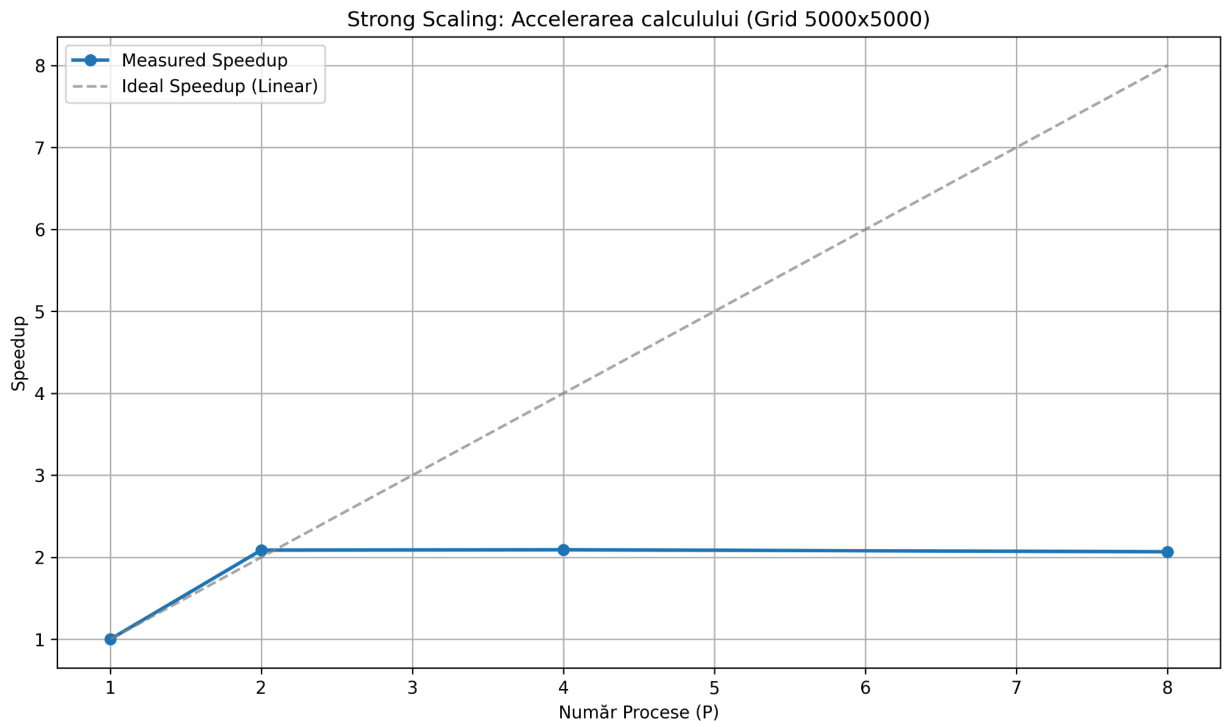
Looking at the results from the overhead graph:

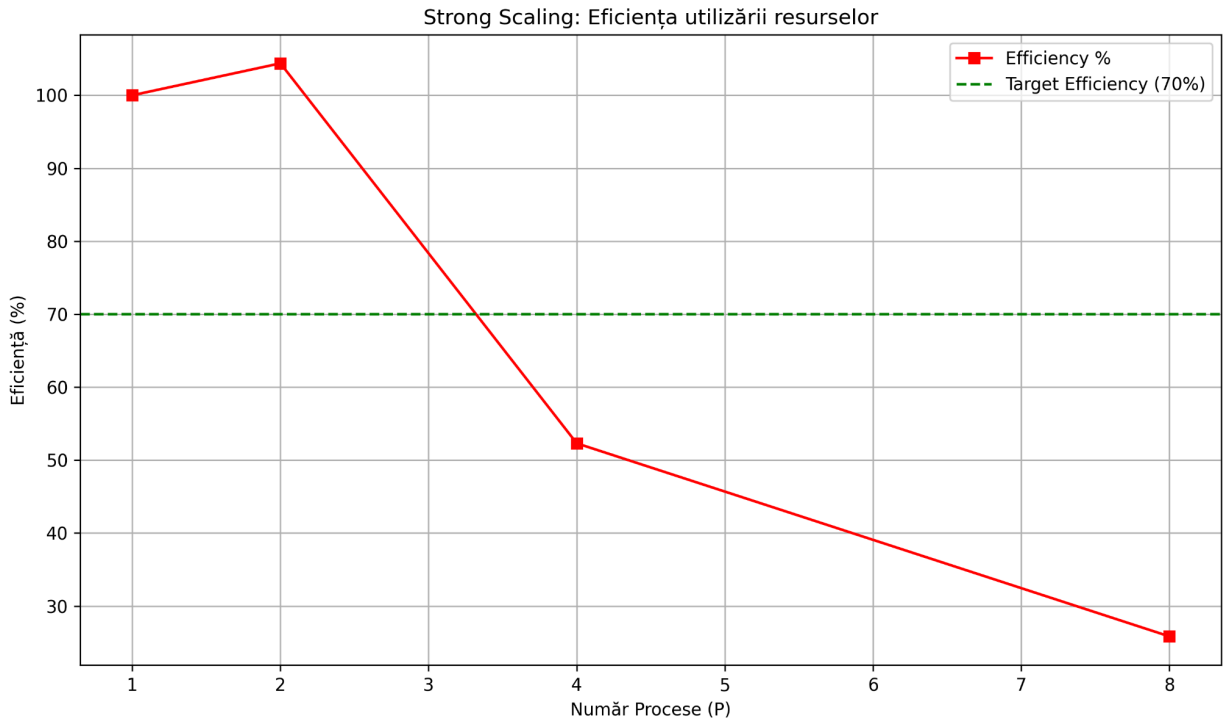
- The pure sequential program, which does not use MPI, runs almost instantly, taking only about two hundredths of a second.
- The MPI runs, using either one or four processes, take over one and painted tenths of a second.
- **Explanation:** This delay represents the fixed startup cost of the MPI runtime. The system requires a set amount of time to initialize network sockets, allocate processes across the cluster, and establish communication channels. For small workloads, this

startup cost completely masks any speed gains from parallel computing. Therefore, MPI is only beneficial for large-scale, long-running simulations where this initialization cost becomes negligible.

## 9.2. Strong Scaling Analysis

In our strong scaling experiment, we kept the total grid size fixed at a large five thousand by five thousand cells and varied the number of processes from one to eight. We analyzed two main metrics: **Speedup** (how many times faster the parallel program runs compared to the sequential one) and **Efficiency** (how well the program utilizes the added processors).





Looking at the strong scaling and efficiency graphs, we can identify two very important behaviors:

### 1. Superlinear Speedup at Two Processes:

- When we use two processes, the speedup is two and eight hundredths, which is higher than the ideal linear speedup of two. This results in an efficiency of one hundred and four percent.
- **The hardware explanation:** This is a classic cache-memory effect. When the entire grid is run on a single process, the dataset is too large to fit into the CPU's fast L3 cache, causing frequent, slow accesses to the main RAM. When we split the grid between two processes, each half of the grid is small enough to fit almost entirely inside the fast cache of its respective processor. The elimination of slow memory retrievals speeds up the execution so much that it easily overcomes the overhead of network communication, pushing the efficiency past one hundred percent.

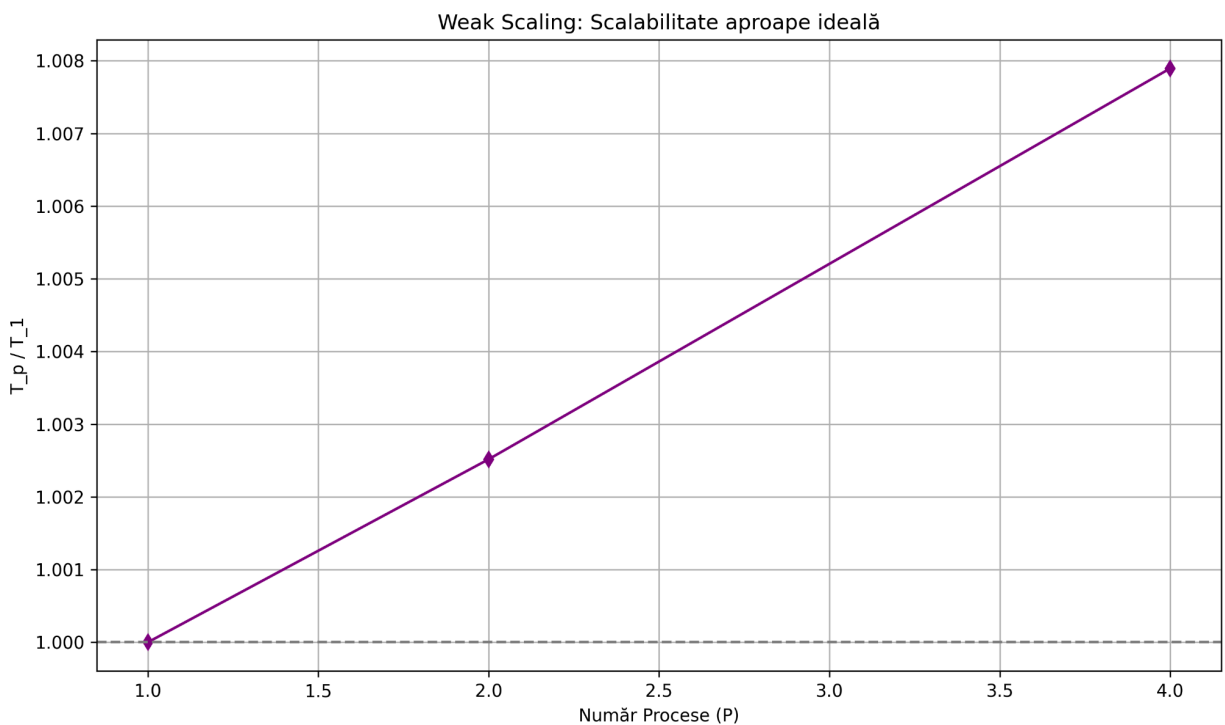
### 2. Performance Bottleneck at Four and Eight Processes:

- When we increase the process count to four, the speedup flattens out, staying at two and ten hundredths (cutting efficiency to about fifty-two percent).
- At eight processes, the speedup drops slightly to two and seven hundredths, pulling efficiency down to just under twenty-six percent.
- **The reason (Amdahl's Law):** As we add more processes to a problem of fixed size, the local grid bands become extremely narrow. The calculation work per process drops, but the communication frequency remains the same. The processes spend more time swapping ghost rows and migrating ants over short

distances than they do calculating the ant steps. Furthermore, collecting the entire grid periodically using gather operations to save progress frames creates a massive bottleneck.

### 9.3. Weak Scaling Analysis

In our weak scaling experiment, we increased the grid size proportionally as we added more processes. This kept the workload per process constant, with each process always managing a five hundred by five hundred subgrid.



Looking at the weak scaling graph:

- At one process, the normalized runtime ratio is exactly one.
- At two processes, the ratio is one and twenty-five ten-thousandths.
- At four processes, the ratio is only one and seventy-nine ten-thousandths (an increase of less than one percent).
- **Explanation:** This is an exceptional result that proves the scalability of our program. It shows that our horizontal slicing strategy and ant migration protocols are highly efficient. As long as we scale our hardware resources in proportion to the data size, the simulation time remains virtually identical. This confirms that the program is fully capable of running massive simulations on large supercomputers.

## 10. Conclusions and Future Work

## Main Conclusions:

1. **One-dimensional horizontal slicing** is highly effective for parallelizing Langton's Ant. It simplifies the communication network since each process only needs to talk to its top and bottom neighbors.
2. **CPU cache locality** plays a massive role in parallel performance. Optimizing the data layout to fit into cache memory resulted in a superlinear speedup of one hundred and four percent at two processes.
3. **Communication overhead** is the primary limit to strong scaling. When the grid bands become too thin, the cost of swapping borders and gathering data dominates the execution time.

## Future Improvements:

- **Two-dimensional Block Partitioning:** Slicing the grid into a checkerboard pattern rather than horizontal bands would reduce the perimeter-to-area ratio of each process's domain. This would lower the amount of border data that needs to be exchanged when running on very high process counts.
- **Dynamic Load Balancing:** If ants cluster in a specific region of the grid, the process managing that region will have a much higher workload. We could implement an algorithm that dynamically adjusts the row boundaries of each process to balance the work evenly.
- **GPU Acceleration:** Offloading the cellular automaton update logic to a graphics processor using CUDA would allow us to simulate millions of ants on massive grids simultaneously.

## 11. References

1. Langton, C. G. (1986). *Studying artificial life with cellular automata*. Physica D: Nonlinear Phenomena, 22(1-3), 120-149.
2. Wolfram, S. (2002). *A New Kind of Science*. Wolfram Media, Champaign, Illinois.
3. Bunimovich, L. A., & Troubetzkoy, S. E. (1992). *Recurrence properties of Lorentz lattice gas cellular automata*. Journal of Statistical Physics, 67(1-2), 289-302.
4. Gajardo, A., Moreira, A., & Goles, E. (2002). *Complexity of Langton's ant*. Discrete Applied Mathematics, 117(1-3), 41-50.
5. Message Passing Interface Forum. (2021). *MPI: A Message-Passing Interface Standard, Version 4.0*.
6. Open MPI Development Team. (2024). *Open MPI Documentation, Version 5.0*.
7. Weisstein, E. W. *Langton's Ant*. MathWorld - A Wolfram Web Resource.
8. Johnston, H. (2012). *Cellular Automata and Langton's Ant Lecture Notes*. University of Massachusetts.